

操作系统 · 第三讲

内存管理： 虚拟内存与页面置换

理解操作系统如何让每个进程以为独占了全部内存——以及当物理内存不够时如何优雅地"换页"。

🕒 预计 45 分钟

📚 先修：进程与线程、C 指针

★ ★ 中等难度

第1章 · 虚拟内存

1.1 虚拟内存

概念

为什么需要

实现原理

1.2 页面置换

1.3 TLB

关键公式

$$EAT = (1 - p) \times T_{\text{mem}} + p \times T_{\text{disk}}$$

虚拟内存 (Virtual Memory)

操作系统为每个进程创建的**独立、连续**的地址空间抽象。进程看到的"地址"（虚拟地址）不是真实的物理内存地址——MMU 负责翻译。

关键洞察：**地址空间 ≠ 物理内存**。32 位系统每个进程"看到" 4GB，但物理内存可能只有 512MB。

地址翻译

$$\text{物理地址} = \text{页表}[\text{虚拟页号}] \times \text{页大小} + \text{页内偏移}$$

$$\text{虚拟页号 (VPN)} = \text{虚拟地址} / \text{页大小} \quad | \quad \text{页内偏移} =$$

$$\text{虚拟地址} \% \text{页大小} \quad | \quad \text{典型页大小} = 4\text{KB}$$

要点

32位地址 + 4KB页 → 20位 VPN (2^{20} =1M 个页表项)。如果每个 PTE 4 字节，单级页表就是 **4MB**——每个进程都要一份！所以需要多级页表。

第1章 · 虚拟内存

▸ 1.1 虚拟内存

概念
▸ 为什么需要
实现原理
1.2 页面置换
1.3 TLB

没有虚拟内存会怎样？三个致命问题

- ✗ 无隔离：进程 A 的野指针可以覆写进程 B 的数据，甚至内核内存
 - ✗ 必须全量加载：游戏 60GB，你的 16GB 电脑根本跑不起来
 - ✗ 外部碎片：频繁分配释放后，空闲空间被切成小块，无法分配大块连续内存
- ✅ 虚拟内存解决全部三个问题：隔离 → 独立地址空间 | 部分加载 → 按需调页 | 碎片 → 物理内存可非连续分配

实际案例：游戏的 60GB 纹理

《最终幻想》安装 60GB。启动时操作系统只加载当前场景需要的纹理页面到 RAM（约 2-3GB），其余留在 SSD。当你走到新区域时触发缺页中断，按需加载。整个过程对游戏透明——它"以为"自己有 60GB 连续内存。

▸ 1.1 虚拟内存

概念
为什么需要
▸ 实现原理
1.2 页面置换
1.3 TLB

📐 页表大小

$$\text{Size} = 2^{32} / 2^{12} \times 4 = 4\text{MB}$$

1 CPU 发出虚拟地址

进程执行 `mov eax, [vaddr]`。vaddr 被拆分为 VPN + offset。

2 MMU 查 TLB

TLB 是页表的硬件缓存（~64-1024 项）。命中 → 直接拿到物理页框号 PFN（~1 cycle）。

3 TLB miss → 页表遍历 (Page Walk)

MMU 从 CR3 寄存器取页表基址，逐级遍历（x86-64 用 4 级页表：PML4→PDPT→PD→PT）。

4 PTE 的 Present 位 = 0 → Page Fault

CPU 陷入内核，OS 的 page fault handler 从磁盘换入页面、更新页表、重试指令。

💡 要点

TLB 覆盖率 = TLB 条目数 × 页大小。64 条 × 4KB = 256KB，但程序的工作集通常远超此值。这就是 TLB miss 不可避免的原因。

页面置换 (Page Replacement)

当物理内存已满、需要加载新页面时，操作系统必须**选择**一个 **victim page** 驱逐到磁盘 (swap out)，为新页面腾出空间。

核心目标：最小化缺页率 → 最大化系统吞吐量。选错 victim 会导致刚换出的页立刻又被访问 (thrashing)。

为什么不能随便选？

选正在用的页面 → 刚换出又换入，磁盘 **I/O** 飙升

理想情况：选 **未来最长时间不会被访问** 的页面 → **Belady** 最优算法 (理论最优，不可实现)

第2章 · 页面置换

1.1 虚拟内存

▸ 1.2 页面置换

概念

▸ 算法对比

Belady 异常

1.3 TLB

算法	策略	缺页率	实现复杂度	硬件支持
OPT (Belady)	淘汰未来最远使用的页	最低（理论）	不可实现	—
LRU	淘汰最久未使用的页	接近 OPT	高（需时间戳）	需硬件计数器
Clock (NRU)	循环扫描，淘汰 reference bit=0 的页	接近 LRU	低	仅需 1 bit/页
FIFO	淘汰最早进入的页	中	最低	无
Random	随机淘汰	不稳定	极低	无

💡 要点

Linux 用的是什么？ 两轮 Clock 算法（类似 LRU 近似）。每个页有两个标志位：PG_referenced 和 PG_active。kswapd 后台线程周期性扫描并降级 inactive 页。

▮ BELADY 异常条件

$$N_{\text{faults}}(M + 1) > N_{\text{faults}}(M)$$

Belady 异常 (Belady's Anomaly)

对 **FIFO** 页面置换算法，增加物理页框数有时反而**增加**缺页次数。这是反直觉的——更多内存应该更好才对。

具体例子：访问序列 **1,2,3,4,1,2,5,1,2,3,4,5**

3 个页框 → 9 次缺页 | **4** 个页框 → **10** 次缺页
(反而更多!)

根因：FIFO 不关心访问频率，4 个页框时可能保留了很多"以后才用"的页面，导致空间利用率反而下降。

💡 要点

LRU 和 **Clock** 没有 **Belady** 异常。因为它们属于"栈算法" (stack algorithm) ——M 个页框时的驻留集始终是 M+1 个页框时的子集。

▮ TLB 命中率影响

$EAT = h \times (T_{tlb} + T_{mem}) + (1 - h) \times (T_{tlb} + 2T_{mem})$

h=TLB 命中率 miss 时多一次内存
访问读页表

TLB (Translation Lookaside Buffer)

MMU 内部的**硬件缓存**，存储最近使用的虚拟页号→物理页框号映射。本质是一个全相联的 SRAM 查找表，通常 64-1024 项。

TLB 下的有效访问时间

$EAT = h \times (T_{tlb} + T_{mem}) + (1 - h) \times (T_{tlb} + 2T_{mem})$

h = TLB 命中率 (通常 > 99%) | $T_{tlb} \approx 1 \text{ cycle}$
| $T_{mem} \approx 100 \text{ cycles}$

💡 要点

h=99% 时, $EAT \approx 1 + 100 = 101 \text{ cycles}$ 。**h=98%** 时, $EAT \approx 1 + 0.98 \times 100 + 0.02 \times 200 = \mathbf{103 \text{ cycles}}$ 。TLB 命中率降 1%，性能损失约 2%。

概念	一句话定义	关键公式
虚拟内存	进程地址空间与物理内存的解耦抽象	—
有效访问时间 (EAT)	考虑缺页后的平均内存访问时间	$EAT = (1 - p)T_{mem} + pT_{disk}$
多级页表	用树结构压缩页表，只为实际使用的区域分配	Size = 4MB/进程 → 按需分配
Page Fault	访问不在物理内存的页面时触发的中断	—
Belady 最优	淘汰未来最远使用的页面——理论下界，不可实现	—
LRU	淘汰最久未使用的页面——最优的可行近似	—
Clock 算法	循环扫描 reference bit，Linux 的实际选择	—
Belady 异常	FIFO 的悖论：更多页框 ≠ 更少缺页	—
TLB	页表缓存——99%+ 命中率是性能关键	$EAT \approx h \cdot 101 + (1 - h) \cdot 201$

Q: 为什么 **Linux** 不用纯 **LRU** 而用 **Clock**?

纯 LRU 需要每次内存访问更新硬件计数器 → 开销太大。Clock 算法只需一个 reference bit（PTE 第 5 位），MMU 硬件自动置位，OS 只需定期扫描清零——近似效果接近 LRU，零额外硬件成本。

Q: 多级页表有什么代价？

每次 TLB miss 需要遍历所有级（x86-64: 4 次内存访问）。所以 TLB 命中率必须极高，否则性能崩塌。这也是大页（2MB/1GB）的价值——减少页表层级。

Q: Belady 异常只在 **FIFO** 中出现吗？

是的。LRU、Clock、OPT 都属于"栈算法"——分配给进程的页框越多，缺页率单调不增。**FIFO** 是唯一不满足栈性质的常见算法。